SIMATS ENGINEERING

CO2_AT2 Reverse Engineering Task

COURSE NAME: Software Testing **COURSE CODE: CSA37**

NAME: M.S.Charishma
REG.NO:192311393

COURSE OUTCOME COVERED

CO2: *Design effective test cases using black-box, white-box testing techniques and to control methods flow.*

Assessment Tool Weightage: 30%

Sl. No	Questions	Bloom's Level
1	A partially tested ATM withdrawal module is provided where only input-output behavior is known (e.g., withdrawal limits, balance validation, error messages). Reverse engineer the system by analyzing the observed behavior and reconstructing the underlying test case design strategy. Decompose the system into functional components and identify possible equivalence classes and boundary conditions. Infer the black box testing techniques used and evaluate their adequacy. Reconstruct an improved set of test cases covering all critical scenarios. Document your analysis with structured explanation.	5
2	A software module's execution traces and code snippets are given, but no test documentation is available. Reverse engineer the white box testing approach by constructing the control flow graph from the given logic. Identify all decision points, loops, and independent execution paths. Evaluate the existing test adequacy in terms of coverage criteria. Reconstruct a complete path testing strategy ensuring full coverage. Present your findings with clear reasoning and structured representation.	5
3	A web application login and transaction system has been tested using unknown techniques, but only defect logs and test results are available. Reverse engineer the testing approach by analyzing defect patterns and identifying which black box and white box techniques were likely used. Decompose the system into testable components and evaluate gaps in testing coverage. Reconstruct a comprehensive testing strategy integrating equivalence partitioning, boundary value analysis, and path testing. Justify your reconstructed approach with logical reasoning and structured documentation.	5

4	A legacy inventory management system provides only a set of successful and failed transaction logs regarding stock updates and low-stock alerts. Reverse engineer the business logic by analyzing these logs to identify the hidden boundary conditions and equivalence partitions. Decompose the update process into functional units and reconstruct a black box test design that addresses the gaps found in the logs. Justify your selection of test cases based on the inferred logic.	5
5	You are provided with a compiled .exe file of a library management module and its corresponding CPU/Memory execution profile during a stress test. Reverse engineer the white box testing requirements by mapping execution spikes to potential loops and decision clusters in the logic. Construct a conceptual control flow graph (CFG) that represents this behavior. Reconstruct a path testing strategy that targets these resource-intensive paths and evaluate its impact on system stability.	6
6	A mobile payment gateway's API responses (JSON) for various transaction states—Pending, Success, Reversal, and Timeout—are given. Reverse engineer the underlying state machine by analyzing the transition triggers and data constraints. Identify the implicit state transition testing technique used. Reconstruct a comprehensive state-based test suite that ensures all legal and illegal transitions are covered. Document your analysis with a structured state-transition diagram.	5
7	An encrypted communication module provides only "black-box" behavior through its input parameters and encrypted outputs. Reverse engineer the validation logic by performing "fuzzing" analysis on the input fields to identify error-handling boundaries. Decompose the module into data validation and encryption components. Reconstruct an improved set of test cases using Boundary Value Analysis (BVA) and justify how your design prevents buffer overflow or injection vulnerabilities.	5
8	A set of legacy unit test results for an insurance premium calculator shows 100% statement coverage but fails to catch defects in multi-condition "if" statements. Reverse engineer the original white box approach and identify the inadequacy in the coverage criteria used. Construct a new CFG and identify all independent execution paths. Reconstruct a path testing strategy that achieves 100% Branch and Path coverage. Explain how this reconstruction reduces the risk of calculation errors.	5

9	A cloud-based file synchronization tool provides user activity logs and a list of "lost file" defect reports. Reverse engineer the synchronization algorithm's logic by analyzing the defect patterns related to file size, type, and network speed. Infer the likely Equivalence Class Partitioning (ECP) used by the original developers. Reconstruct a revised testing strategy that integrates BVA with stress testing. Justify how your reconstructed approach prevents future data loss.	5
10	You are given a sequence of database queries (SQL) generated by an e-commerce "Search and Filter" module, but no source code. Reverse engineer the search logic by analyzing the query parameters and the resulting dataset. Identify the decision-making logic used for complex filtering (e.g., AND/OR/NOT). Reconstruct a Decision Table based testing strategy that covers all possible search combinations. Evaluate the completeness of your reconstructed model.	5

Code of Conduct:

I _M.S.Charishma___ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: *M.S.Charishma*

RUBRICS FOR CALCULATION:3

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of System	20%	Demonstrates clear and in-depth understanding of the system/product and its functionality	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Lacks understanding of the system/product
Decomposition	25%	Effectively breaks down the system into components with detailed analysis	Decomposes system with minor gaps in analysis	Limited or incomplete decomposition	Unable to analyze or break down system
Identification of Components	20%	Accurately identifies all components and explains their functions clearly	Identifies most components with minor errors	Identifies few components; explanations are weak	Unable to identify components or functions
Reconstruction	20%	Successfully reconstructs or models the system with accuracy and logic	Reconstruction is mostly correct with minor issues	Partial reconstruction; lacks clarity	Unable to reconstruct or model system
Documentation	15%	Well-organized, clear documentation with diagrams and structured explanation	Documentation is adequate with minor issues	Poorly organized or incomplete documentation	No proper documentation or presentation

ANSWERS

1. ATM Withdrawal Module

The ATM can be reverse engineered by observing inputs, outputs, and error messages. Its main functional components are card/PIN validation, account balance checking, withdrawal-limit checking, cash availability, transaction approval, and receipt or error generation.

Equivalence classes include: valid amount within the allowed range; amount below the minimum; amount above the maximum; amount not in a permitted denomination; amount greater than the available balance; invalid PIN/card; and ATM cash unavailable. Boundary Value Analysis should test values just below, at, and just above the minimum and maximum limits. For example, if the permitted range is ₹500–₹25,000, test ₹499, ₹500, ₹501, ₹24,999, ₹25,000, and ₹25,001, along with balance and denomination checks.

An improved test set should cover valid withdrawal, zero and negative amounts, insufficient balance, daily-limit exceeded, invalid PIN attempts, unavailable cash, network failure, and cancellation. Each case should record preconditions, input, expected message, account balance before/after, and actual result. The inferred original approach likely checked ordinary valid inputs but missed boundaries and combinations. Combining ECP, BVA, and a small decision table gives better coverage with fewer redundant tests. Random testing can supplement this suite but should not replace systematic boundary and negative tests.

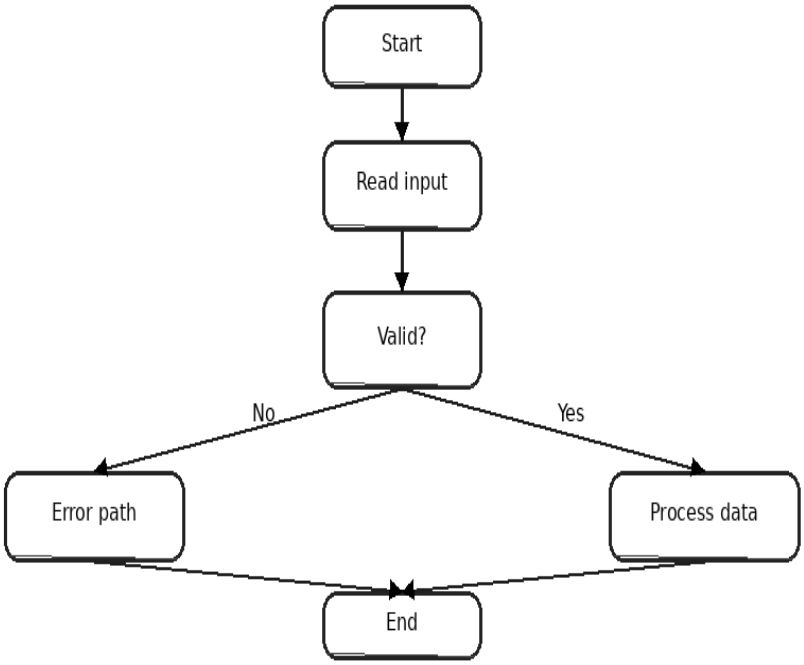
2. Reverse Engineering White-Box Testing

First, read the supplied execution traces and code snippets to identify each statement, decision, and loop. Since the exact code is not included in the scenario, the following is a conceptual control-flow model rather than a claim about the unseen implementation. The main decision is whether the input is valid. A valid input follows the processing path; an invalid input follows the error path. If a loop exists in the given code, its zero-iteration, one-iteration, and repeated-iteration behavior must also be tested.

The current test adequacy cannot be confirmed without the original test results. Statement coverage only proves that statements executed; it does not prove that every decision outcome or independent path was exercised. Branch coverage requires both True and False outcomes for each decision. Path testing should identify independent paths and exercise each feasible path, including error handling and loop boundaries.

Use the conceptual CFG below to organize the reconstructed suite. Add test cases for valid input, invalid input, each decision outcome, empty data, one record, multiple records, and loop termination. Calculate cyclomatic complexity using $V(G) = E - N + 2P$ (for one connected flow graph, $P = 1$), then select a basis set of independent paths. Document any infeasible paths and the reason they cannot execute. This provides traceable evidence of coverage and reduces missed logic defects.

Rough control-flow sketch: example module



3. Login and Transaction System

The system should be divided into login validation, session creation, transaction input validation, balance or limit checking, transaction processing, and result logging. Defect patterns help infer earlier techniques: failures at minimum or maximum values suggest incomplete BVA; failures for invalid formats suggest weak ECP; and defects occurring only after a particular decision sequence suggest missing branch or path tests.

For login, test valid credentials, invalid username, invalid password, empty fields, locked account, expired session, and repeated failed attempts. For transactions, test valid amounts, zero, negative values, values at and around transaction limits, insufficient balance, duplicate submission, timeout, and cancellation. Use ECP to group similar inputs and BVA to focus on edges.

White-box path testing should examine success and failure branches, authentication before transaction processing, and exception/timeout paths. A hybrid test matrix should link every requirement to test inputs, expected results, observed defects, and coverage evidence. Regression tests should be run after fixes. Because source code and actual defect logs are not supplied in the scenario, the exact original technique cannot be proven; the proposed strategy is reconstructed from the defect patterns described. It gives more complete functional and logic coverage than relying on a single testing technique.

4. Inventory Management System

The logs should be grouped by stock-update result and low-stock alert behavior. The functional units are input validation, stock addition/removal, stock total calculation, threshold comparison, alert

generation, and transaction recording. From the logs, infer the stock values at which behavior changes and compare successful and failed records to identify likely boundaries.

Equivalence partitions include positive stock updates, zero quantity, negative quantity, quantity exceeding available stock, valid stock above the alert threshold, stock equal to the threshold, and stock below the threshold. If the alert is triggered when stock is less than or equal to a threshold T, test T−1, T, and T+1. Also test a stock update that changes the level from above the threshold to below it, and a repeated update that might create a duplicate alert.

The reconstructed black-box suite should include normal receipt, normal issue, insufficient-stock issue, invalid quantity, boundary threshold values, repeated transactions, and database or network failure. Expected stock totals and alert states should be specified for every case. Logs alone may not reveal the exact hidden implementation, so inferred rules must be marked as assumptions and confirmed with the system owner. This approach makes the test design evidence-based and exposes gaps in the original transaction tests.

I

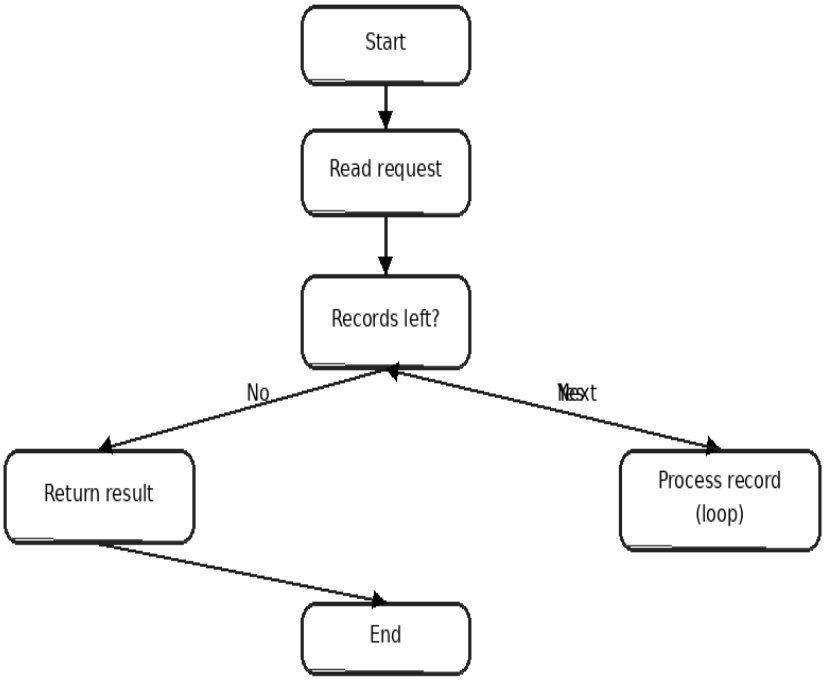
5. Library Management .exe and Resource Profile

A compiled executable does not reveal the source-level control flow directly. CPU and memory spikes can indicate repeated record processing, nested loops, large sorting/search operations, memory allocation, or an error-retry loop. These are hypotheses that must be confirmed through profiling, input-size comparisons, and observable behavior; a spike alone does not prove a particular code structure.

A conceptual CFG begins with request validation, proceeds to record loading, checks whether records remain, processes each record, and repeats until the loop finishes. Additional branches should represent invalid requests, missing records, and resource or I/O errors. The sketch below illustrates one possible loop, not the exact unseen program.

Construct tests using an empty library, one record, a typical collection, a very large collection, duplicate records, missing records, and malformed requests. Measure execution time and peak memory for each size. Path tests should target loop entry, loop exit, repeated iteration, and exception handling. Stress tests should include sustained requests and concurrent access where supported. Set practical response-time and memory limits, and check that the program terminates without data corruption. This combines reverse engineering from behavior with targeted path and performance testing.

Rough CFG: resource-heavy library operation



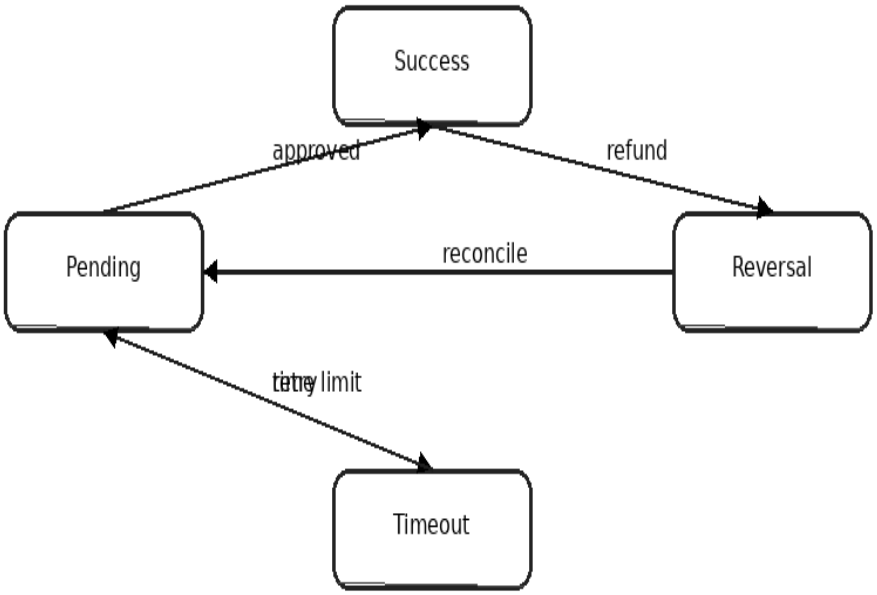
6. Mobile Payment Gateway State Machine

The observed states are Pending, Success, Reversal, and Timeout. A state-transition test suite should test both the state and the event that causes the transition. A conceptual model is shown below; actual transitions must be verified against the API contract because the prompt does not provide the specific JSON responses or transition rules.

Possible legal transitions include Pending → Success after approval, Pending → Timeout when the time limit expires, and Success → Reversal when a reversal is accepted. A Timeout → Pending retry may be legal only if the API supports retry. Repeated success callbacks, reversal before success, duplicate requests, and late success after timeout should be tested as potentially illegal or specially handled transitions.

For every transition, verify the HTTP status, response code, transaction ID, amount, timestamp, and final transaction status. Test missing or malformed fields, duplicate callbacks, delayed responses, network interruption, and idempotent retry behavior. Confirm that a transaction is not credited twice and that a reversal updates balances consistently. The suite should cover every legal transition and representative illegal transitions, including attempts to skip required states. The transition diagram is a reconstructed model and should be updated when the documented API behavior is available.

Rough state-transition sketch: payment gateway



7. Encrypted Communication Module

The module should be separated into input parsing and validation, length and format checks, encryption, and output/error handling. Fuzzing means supplying unusual, malformed, oversized, empty, and unexpected input values while observing whether the system rejects them safely or produces incorrect output. Since encryption is a black box here, tests should focus on externally observable behavior and documented input constraints rather than assuming the internal algorithm.

Use BVA around each documented length limit: minimum valid length, one below minimum, maximum valid length, and one above maximum. Include empty strings, very long strings, invalid encodings, unexpected characters, null or missing fields, repeated delimiters, and malformed structured input. Verify that invalid data is rejected with a controlled error and does not crash the program, expose sensitive data, or corrupt memory.

Security testing should also check that inputs cannot bypass validation, that error messages do not reveal keys or plaintext, and that valid input produces output that can be correctly decrypted by an authorized counterpart where applicable. Boundary tests can reveal truncation and buffer-handling weaknesses, while fuzzing explores combinations that hand-written tests may miss. Testing reduces risk but cannot by itself guarantee the absence of buffer overflow or injection vulnerabilities; code review and appropriate security analysis are also needed.

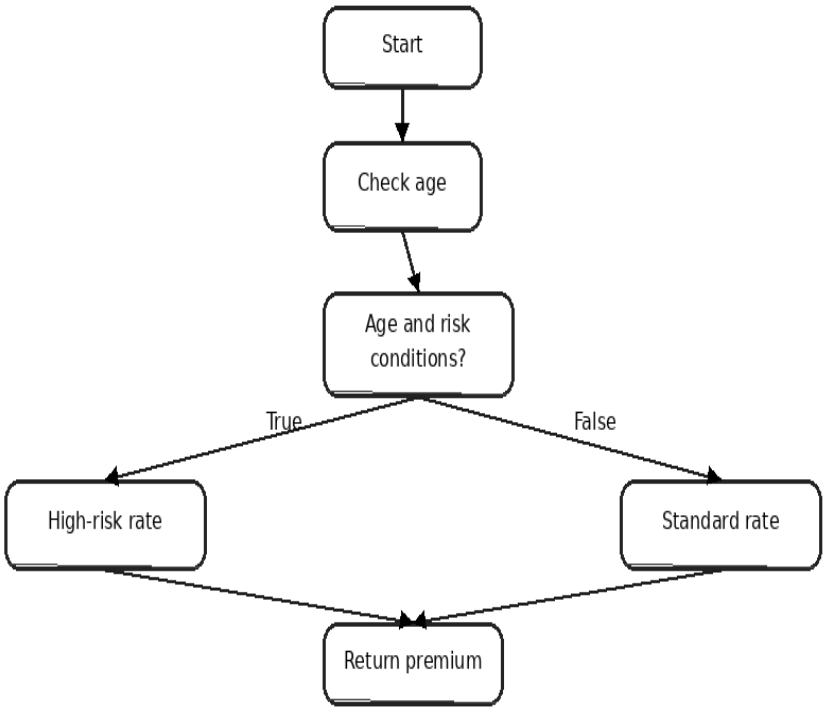
8. Insurance Premium Calculator

One hundred percent statement coverage means each executable statement ran at least once, but it does not guarantee that each Boolean condition or decision outcome was independently tested. For example, a condition such as “age is above the limit AND risk category is high” may execute its calculation statement while still missing combinations where only one condition changes.

Reconstruct a CFG with a start node, input checks, a compound decision, separate high-risk and standard-rate branches, and a final premium output. The sketch below is conceptual because the actual source code is not included. Identify every decision and each feasible branch. Use branch coverage to test True and False outcomes, and condition coverage to make each individual Boolean condition True and False.

For a decision A AND B, test (A=True, B=True), (A=False, B=True), and (A=True, B=False), along with any additional combinations required by the actual logic. Add boundary tests for age, policy value, discount, and premium limits. Basis Path Testing should select independent feasible paths based on cyclomatic complexity. Verify calculated premiums against independently calculated expected values and test invalid or missing inputs. This exposes logic errors that statement coverage can miss. Claiming 100% path coverage is only reasonable for a finite, feasible path set; loops or complex combinations may make exhaustive path coverage impractical.

Rough CFG: premium calculation conditions



9. Cloud File Synchronization

The defect reports should be grouped by file size, file type, network speed, and the action occurring when the file was lost. These patterns can reveal missing equivalence classes or boundaries, but the exact original ECP cannot be known without the source code and detailed logs.

Useful file-size classes are empty files, ordinary files, files near the supported maximum, and files exceeding the maximum. File-type classes include supported formats, unsupported formats, extension/content mismatch, and files with unusual names. Network classes include stable high speed, slow connection, intermittent connection, and disconnection during upload or download.

Combine BVA with stress and recovery testing. Test sizes immediately below, at, and above the documented limit; many small files; a few very large files; simultaneous changes on two devices; interrupted transfers; low disk space; and restart during synchronization. Check checksums or hashes before and after transfer, verify that no file is silently overwritten, and confirm conflict

handling and retry behavior. Preserve logs and repeat the same scenario to verify reproducibility. Stress testing should increase file count and concurrency gradually while monitoring errors and memory. These tests address both input boundaries and environmental conditions that may cause data loss.

10. E-commerce Search and Filter Queries

The SQL queries and resulting datasets can be compared to infer the filtering rules. Identify each parameter, its default value, how it is combined with other parameters, and whether the intended relationship is AND, OR, or NOT. For example, a keyword and category may be combined with AND so that both must match; alternatively, multiple selected categories may be combined with OR. These rules must be confirmed from observed outputs because the source code is unavailable.

A reconstructed decision table should include conditions such as keyword provided (Yes/No), category selected (Yes/No), price range selected (Yes/No), and availability filter selected (Yes/No). The actions are to apply the relevant filters and display the resulting dataset. Test no filters, each filter alone, every important pair of filters, all filters together, empty result sets, invalid ranges, and contradictory criteria. Also test special characters, empty strings, pagination, and sorting if they are part of the interface.

Compare each returned dataset with an independently calculated expected result. Confirm that the query does not accidentally broaden results due to incorrect OR precedence or exclude valid records due to incorrect AND precedence. Use representative combinations and remove redundant cases only when they exercise the same rule. The decision-flow sketch below illustrates a simple keyword/category path; the final decision table should reflect the actual query behavior.

Rough decision flow: search and filter

